**Algorithm 1** Model Architecture: CLIP ViT-L/14 with LoRA Adaptation

**Require:** Pretrained CLIP ViT-L/14 $\Phi$; target modules $\mathcal{T} = \{\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v, \mathbf{W}_o\}$; LoRA ranks $r_{\text{attn}} = 4$, $r_{\text{head}} = 2$; LoRA scales $\alpha_{\text{attn}} = 16$, $\alpha_{\text{head}} = 8$; flag `full_train_head` (True = all head params trainable, False = LoRA head)

**Ensure:** Trainable detector $F(\cdot) = g_\phi \circ f_\theta(\cdot)$

**// Step 1: Freeze backbone**
**for** $\mathbf{W} \in \Phi$.parameters() **do**
    $\mathbf{W}$.requires_grad $\leftarrow$ False
**end for**
**// Step 2: Inject LoRA into attention projections**
**for** each module $\mathbf{W} \in \Phi$.vision_model.modules() **do**
    **if** $\mathbf{W}$ is Linear $\wedge$ name($\mathbf{W}$) $\in \mathcal{T}$ **then**
        $\mathbf{A} \sim \mathcal{N}(0, 0.02^2) \in \mathbb{R}^{d_{\text{in}} \times r_{\text{attn}}}$
        $\mathbf{B} \leftarrow \mathbf{0} \in \mathbb{R}^{r_{\text{attn}} \times d_{\text{out}}}$
        **// Copy pretrained weights into new LoRA layer, then replace**
        $\widetilde{\mathbf{W}}$.weight $\leftarrow \mathbf{W}$.weight;  $\widetilde{\mathbf{W}}$.bias $\leftarrow \mathbf{W}$.bias
        Replace $\mathbf{W}$ with $\widetilde{\mathbf{W}}(\mathbf{x}) = \mathbf{W}\mathbf{x} + \frac{\alpha_{\text{attn}}}{r_{\text{attn}}} \cdot (\mathbf{x}\mathbf{A})\mathbf{B}$
        $\mathbf{W}$.requires_grad $\leftarrow$ False;  $\mathbf{A}, \mathbf{B}$.requires_grad $\leftarrow$ True
    **end if**
**end for**
**// Step 3: Construct classifier head**
$\mathbf{W}_h \sim$ Kaiming_uniform $\in \mathbb{R}^{2 \times 1024}$
$\mathbf{b}_h \sim \mathcal{U}(-\text{bound}, \text{bound}) \in \mathbb{R}^2$
**if** `full_train_head` **then**
    $\mathbf{W}_h$.requires_grad $\leftarrow$ True;  $\mathbf{b}_h$.requires_grad $\leftarrow$ True
    $g_\phi(\mathbf{z}) \leftarrow \mathbf{W}_h\mathbf{z} + \mathbf{b}_h$ ▷ All 2050 params trainable
**else**
    $\mathbf{W}_h$.requires_grad $\leftarrow$ False;  $\mathbf{b}_h$.requires_grad $\leftarrow$ False
    **// LoRA factors (trainable):** $\mathbf{A}_h \in \mathbb{R}^{1024 \times r_{\text{head}}}$, $\mathbf{B}_h \in \mathbb{R}^{r_{\text{head}} \times 2}$
    $\mathbf{A}_h \sim \mathcal{N}(0, 0.02^2)$;  $\mathbf{B}_h \leftarrow \mathbf{0}$
    $g_\phi(\mathbf{z}) = \underbrace{\mathbf{W}_h\mathbf{z} + \mathbf{b}_h}_{\text{frozen base}} + \underbrace{\frac{\alpha_{\text{head}}}{r_{\text{head}}} \cdot (\mathbf{z}\mathbf{A}_h)\mathbf{B}_h}_{\text{trainable LoRA}} \in \mathbb{R}^2$
**end if**
**// Step 4: Full forward pass**
$f_\theta(\mathbf{x}) = \Phi_{\text{LoRA}}(\mathbf{x})$.pooler_output $\in \mathbb{R}^{1024}$ ▷ ViT output: CLS token after LoRA attention
$\hat{y} = \text{softmax}(g_\phi(f_\theta(\mathbf{x})))_1$ ▷ Predicted probability of being fake
**return** $F(\cdot) = g_\phi \circ f_\theta$

**Algorithm 2** BalanceBatchSampler (v1): Class-Balanced Mini-Batch Construction

**Require:** Dataset labels $\mathbf{y} \in \{0,1\}^N$ (0 = real, 1 = fake); Batch-size control $b \in \mathbb{N}^+$ (total batch $\approx 2b$); Real ratio $\rho \in (0,1)$ (proportion of real samples per batch, default 0.5); Shuffle flag $s \in \{\text{True}, \text{False}\}$

**Ensure:** Sequence of mini-batches $\{\mathcal{B}_t\}_{t=0}^{M-1}$, $|\mathcal{B}_t| = n_r + n_f$ where $n_r/n_f$ follows $\rho$

$\mathcal{I}_0 \leftarrow \{i \mid \mathbf{y}[i] = 0\}$; $\mathcal{I}_1 \leftarrow \{i \mid \mathbf{y}[i] = 1\}$

$n_r \leftarrow \max(1, \lfloor 2b \cdot \rho \rceil)$; $n_f \leftarrow 2b - n_r$

$M \leftarrow \min\left(\lfloor |\mathcal{I}_0| \,/\, n_r \rfloor,\ \lfloor |\mathcal{I}_1| \,/\, n_f \rfloor\right)$

**if** $M = 0$ **then**

    **raise** ValueError (insufficient samples for $n_r + n_f$)

**end if**

**// Step 1: Shuffle indices within each class independently**

**if** $s$ **then**

    $\mathcal{I}_0 \leftarrow \text{Shuffle}(\mathcal{I}_0)$; $\mathcal{I}_1 \leftarrow \text{Shuffle}(\mathcal{I}_1)$

**end if**

**// Step 2: Construct batches**

$\mathbb{B} \leftarrow [\,]$

**for** $t = 0$ **to** $M - 1$ **do**

    $\mathcal{B}_t \leftarrow [\,]$

    $\mathcal{B}_t$.extend$\big(\mathcal{I}_0[t \cdot n_r : (t+1) \cdot n_r]\big)$ ▷ $n_r$ reals

    $\mathcal{B}_t$.extend$\big(\mathcal{I}_1[t \cdot n_f : (t+1) \cdot n_f]\big)$ ▷ $n_f$ fakes

    $\mathbb{B}$.append$(\mathcal{B}_t)$

**end for**

**// Step 3: Shuffle batch order**

**if** $s$ **then**

    $\mathbb{B} \leftarrow \text{Shuffle}(\mathbb{B})$

**end if**

**return** $\mathbb{B}$

**Key parameter:** $\rho = 0.5$ yields equal real/fake (baseline);

$\rho = 0.7$–$0.85$ yields high-real-image ratios (more RR + RF outputs).

**Limitation:** single-GPU only (DDP not yet supported).

**Algorithm 3** BalancePairSampler (v2): Explicit (Real, Fake) Pair Construction

**Require:** Dataset labels $\mathbf{y} \in \{0,1\}^N$ (0 = real, 1 = fake); Pairs per batch $P \in \mathbb{N}^+$; Shuffle flag $s \in \{\text{True}, \text{False}\}$

**Ensure:** Sequence of interleaved batches $\{\mathcal{B}_t\}_{t=0}^{M-1}$, each $\mathcal{B}_t = [r_1, f_1, r_2, f_2, \ldots, r_P, f_P]$ (length $2P$)

$\mathcal{I}_0 \leftarrow \{i \mid \mathbf{y}[i] = 0\}$; $\mathcal{I}_1 \leftarrow \{i \mid \mathbf{y}[i] = 1\}$
$M \leftarrow \lfloor \min(|\mathcal{I}_0|, |\mathcal{I}_1|) / P \rfloor$
**if** $M = 0$ **then**
    **raise** ValueError (insufficient samples for $P$ pairs)
**end if**
**// Step 1: Shuffle intra-class indices independently**
**if** $s$ **then**
    $\mathcal{I}_0 \leftarrow \text{Shuffle}(\mathcal{I}_0)$; $\mathcal{I}_1 \leftarrow \text{Shuffle}(\mathcal{I}_1)$
**end if**
**// Step 2: Build interleaved batches**
$\mathbb{B} \leftarrow [\,]$
**for** $t = 0$ **to** $M - 1$ **do**
    $\mathcal{B}_t \leftarrow [\,]$
    **for** $p = 0$ **to** $P - 1$ **do**
        $\mathcal{B}_t$.append$\big(\mathcal{I}_0[t \cdot P + p]\big)$ ▷ real
        $\mathcal{B}_t$.append$\big(\mathcal{I}_1[t \cdot P + p]\big)$ ▷ fake
    **end for**
    $\mathbb{B}$.append$(\mathcal{B}_t)$
**end for**
**// Step 3: Shuffle batch order**
**if** $s$ **then**
    $\mathbb{B} \leftarrow \text{Shuffle}(\mathbb{B})$
**end if**
**return** $\mathbb{B}$

**Key property:** Every batch element is an explicit (real, fake) pair.
When used with RFPairLapPyramidMixup (Alg. 5),
only RF-mixed outputs are produced — no RR, FF, or FR waste.
**Limitation:** single-GPU only (DDP not yet supported).

**Algorithm 4** LapPyramidMixup (`mixup_mode = lap_pyramid`, with v1 sampler)

**Require:** Batch $\{(\mathbf{x}_i, y_i)\}_{i=1}^{B}$ with $\mathbf{x}_i \in \mathbb{R}^{C\times H\times W}$, $y_i \in \{0,1\}$ (0 = real, 1 = fake); $\alpha > 0$ (Beta shape, default 5); $\gamma \geq 1$ (asymmetry exponent, default 1); $K \in \mathbb{N}^+$ (pyramid levels, default 3); $\boldsymbol{\omega} \in \mathbb{R}^K$ (level weights, default $\omega_k = (K-k)\,/\,\sum_j (K-j)$); $\varepsilon = 10^{-8}$ (numerical stability)

**Ensure:** Mixed batch $\{(\tilde{\mathbf{x}}_i, \tilde{y}_i, y_i^{\text{hard}})\}_{i=1}^{B}$ where $\tilde{\mathbf{x}}_i \in \mathbb{R}^{C\times H\times W}$, $\tilde{y}_i \in [0,1]$, $y_i^{\text{hard}} \in \{0,1\}$

$\lambda \sim \text{Beta}(\alpha,\alpha)$; $\quad q \leftarrow 1-\lambda$ ▷ Fake residual injection strength
$\pi \leftarrow \text{RandPerm}(\{1,\dots,B\})$
**// Step 1: Classify pair types (Q1 anchor rule: real always structural anchor)**
$\mathcal{RR} \leftarrow \{i \mid y_i = 0 \wedge y_{\pi(i)} = 0\}$; $\quad n_{rr} \leftarrow |\mathcal{RR}|$
$\mathcal{FF} \leftarrow \{i \mid y_i = 1 \wedge y_{\pi(i)} = 1\}$; $\quad n_{ff} \leftarrow |\mathcal{FF}|$
$\mathbf{r} \leftarrow$ real anchor positions in cross-class pairs (merge FR into RF by swapping)
$\mathbf{f} \leftarrow$ corresponding fake partner positions
$n_{rf} \leftarrow |\mathbf{r}|$
**// Step 2: RR — pixel-space mixup, hard label 0**
$\tilde{\mathbf{X}}_{rr} \leftarrow \lambda \cdot \mathbf{X}[\mathcal{RR}] + (1-\lambda)\cdot \mathbf{X}[\pi(\mathcal{RR})]$
$\tilde{\mathbf{y}}_{rr} \leftarrow \mathbf{0}^{n_{rr}}$
**// Step 3: FF — pixel-space mixup, hard label 1**
$\tilde{\mathbf{X}}_{ff} \leftarrow \lambda \cdot \mathbf{X}[\mathcal{FF}] + (1-\lambda)\cdot \mathbf{X}[\pi(\mathcal{FF})]$
$\tilde{\mathbf{y}}_{ff} \leftarrow \mathbf{1}^{n_{ff}}$
**// Step 4: RF — Laplacian pyramid residual mixup**
**if** $n_{rf} > 0$ **then**
  $\mathbf{X}_r \leftarrow \mathbf{X}[\mathbf{r}]$; $\quad \mathbf{X}_f \leftarrow \mathbf{X}[\mathbf{f}]$ ▷ Anchors, partners
  **for** $i = 1$ **to** $n_{rf}$ **do** ▷ Element-wise pairs $(\mathbf{r}[i], \mathbf{f}[i])$
    $\mathbf{x}_r \leftarrow \mathbf{X}_r[i]$; $\quad \mathbf{x}_f \leftarrow \mathbf{X}_f[i]$
    **// (a) Build Gaussian pyramids (5-tap binomial [1,4,6,4,1]/16, separable, then $\downarrow 2$)**
    $\mathbf{G}_0^r \leftarrow \mathbf{x}_r$; $\quad \mathbf{G}_0^f \leftarrow \mathbf{x}_f$
    **for** $\ell = 0$ **to** $K-1$ **do**
      $\mathbf{G}_{\ell+1} \leftarrow \text{PyrDown}(\mathbf{G}_\ell)$
    **end for**
    **// (b) Build Laplacian pyramids: $\mathbf{L}_\ell = \mathbf{G}_\ell - \text{PyrUp}(\mathbf{G}_{\ell+1})$**
    // (PyrUp = bilinear upsampling $\uparrow 2$, `F.interpolate(mode='bilinear')`)
    **for** $\ell = 0$ **to** $K-1$ **do**
      $\mathbf{L}_\ell^r \leftarrow \mathbf{G}_\ell^r - \text{PyrUp}(\mathbf{G}_{\ell+1}^r)$
      $\mathbf{L}_\ell^f \leftarrow \mathbf{G}_\ell^f - \text{PyrUp}(\mathbf{G}_{\ell+1}^f)$
    **end for**
    **// (c) Mix Laplacian residuals level-by-level**
    **for** $\ell = 0$ **to** $K-1$ **do**
      $\mathbf{L}_\ell^{\text{mix}} \leftarrow (1-q)\cdot \mathbf{L}_\ell^r + q \cdot \mathbf{L}_\ell^f$
      $E_\ell^r \leftarrow \|\mathbf{L}_\ell^r\|_F^2$; $\quad E_\ell^f \leftarrow \|\mathbf{L}_\ell^f\|_F^2$
    **end for**
    **// (d) Fake evidence from residual energy ratio**
    $\text{num} \leftarrow q^2 \sum_{\ell=0}^{K-1} \omega_\ell \cdot E_\ell^f$
    $\text{den} \leftarrow \sum_{\ell=0}^{K-1} \omega_\ell \left[(1-q)^2 \cdot E_\ell^r + q^2 \cdot E_\ell^f\right] + \varepsilon$
    $e_f \leftarrow \text{num} \,/\, \text{den}$
    **// (e) Reconstruct: coarse structure from real + mixed residuals**
    $\tilde{\mathbf{x}} \leftarrow \mathbf{G}_K^r$ ▷ Coarsest Gaussian of real anchor
    **for** $\ell = K-1$ **downto** $0$ **do**
      $\tilde{\mathbf{x}} \leftarrow \text{PyrUp}(\tilde{\mathbf{x}}) + \mathbf{L}_\ell^{\text{mix}}$
    **end for**
    $\tilde{\mathbf{x}} \leftarrow \text{clamp}(\tilde{\mathbf{x}},\ \min(\mathbf{x}_r),\ \max(\mathbf{x}_r))$
    **// (f) Energy-grounded soft label (Aggressively Fake)**
    $\tilde{y} \leftarrow 1 - (1-e_f)^\gamma$
  **end for**
**end if**
**// Step 5: Assemble final batch**
$\tilde{\mathbf{X}} \leftarrow \text{Concat}(\tilde{\mathbf{X}}_{rr}, \tilde{\mathbf{X}}_{ff}, \tilde{\mathbf{X}}_{rf})$
$\tilde{\mathbf{y}} \leftarrow \text{Concat}(\tilde{\mathbf{y}}_{rr}, \tilde{\mathbf{y}}_{ff}, \tilde{\mathbf{y}}_{rf})$
$\mathbf{y}^{\text{hard}} \leftarrow \text{Concat}(\mathbf{0}^{n_{rr}}, \mathbf{1}^{n_{ff}}, \mathbf{0}^{n_{rf}})$
**return** $\{(\tilde{\mathbf{x}}_i, \tilde{y}_i, y_i^{\text{hard}})\}$

**Algorithm 5** RFPairLapPyramidMixup (v2 sampler, RF-only)

**Require:** Interleaved batch $\{(\mathbf{x}_i, y_i)\}_{i=1}^{2N}$ from BalancePairSampler with pairs $(r_k, f_k)_{k=1}^{N}$; $\alpha > 0$; $\gamma \geq 1$; $K \in \mathbb{N}^+$; $\boldsymbol{\omega}$; $\varepsilon$ (same as Alg. 4)

**Ensure:** Mixed batch $\{(\tilde{\mathbf{x}}_k, \tilde{y}_k, y_k^{\text{hard}})\}_{k=1}^{N}$ (N outputs from 2N inputs)

$\lambda \sim \text{Beta}(\alpha, \alpha)$; $\quad q \leftarrow 1 - \lambda$

$\mathbf{X}_r \leftarrow \mathbf{X}[0 :: 2]$ ▷ Real anchors $[N, C, H, W]$

$\mathbf{X}_f \leftarrow \mathbf{X}[1 :: 2]$ ▷ Fake partners $[N, C, H, W]$

**// No pair-type classification needed — every pair is (real, fake)**

**// Gaussian pyramids (batch-parallel)**

$\mathbf{G}_0^r \leftarrow \mathbf{X}_r$; $\quad \mathbf{G}_0^f \leftarrow \mathbf{X}_f$

**for** $\ell = 0$ **to** $K - 1$ **do**

  $\mathbf{G}_{\ell+1}^r \leftarrow \text{PyrDown}(\mathbf{G}_\ell^r)$

  $\mathbf{G}_{\ell+1}^f \leftarrow \text{PyrDown}(\mathbf{G}_\ell^f)$

**end for**

**// Laplacian pyramids**

**for** $\ell = 0$ **to** $K - 1$ **do**

  $\mathbf{L}_\ell^r \leftarrow \mathbf{G}_\ell^r - \text{PyrUp}(\mathbf{G}_{\ell+1}^r)$

  $\mathbf{L}_\ell^f \leftarrow \mathbf{G}_\ell^f - \text{PyrUp}(\mathbf{G}_{\ell+1}^f)$

**end for**

**// Mix Laplacian residuals level-by-level**

**for** $\ell = 0$ **to** $K - 1$ **do**

  $\mathbf{L}_\ell^{\text{mix}} \leftarrow (1 - q) \cdot \mathbf{L}_\ell^r + q \cdot \mathbf{L}_\ell^f$

  $E_\ell^r \leftarrow \text{per-sample } \|\mathbf{L}_\ell^r\|_F^2 \in \mathbb{R}^N$; $\quad E_\ell^f \leftarrow \text{per-sample } \|\mathbf{L}_\ell^f\|_F^2 \in \mathbb{R}^N$

**end for**

**// Fake evidence (per-sample energy ratio)**

$\text{num} \leftarrow q^2 \sum_{\ell=0}^{K-1} \omega_\ell \cdot E_\ell^f$

$\text{den} \leftarrow \sum_{\ell=0}^{K-1} \omega_\ell \left[(1 - q)^2 \cdot E_\ell^r + q^2 \cdot E_\ell^f\right] + \varepsilon$

$\mathbf{e}_f \leftarrow \text{num} \,/\, \text{den} \in \mathbb{R}^N$

**// Reconstruct from coarse real + mixed residuals**

$\tilde{\mathbf{X}} \leftarrow \mathbf{G}_K^r$ ▷ Start from real's coarsest Gaussian

**for** $\ell = K - 1$ **downto** $0$ **do**

  $\tilde{\mathbf{X}} \leftarrow \text{PyrUp}(\tilde{\mathbf{X}}) + \mathbf{L}_\ell^{\text{mix}}$

**end for**

$\tilde{\mathbf{X}} \leftarrow \text{clamp}(\tilde{\mathbf{X}},\ \min(\mathbf{X}_r),\ \max(\mathbf{X}_r))$ ▷ Per-sample clamping

**// Soft label (all real anchors)**

$\tilde{\mathbf{y}} \leftarrow 1 - (1 - \mathbf{e}_f)^\gamma \in [0, 1]^N$

$\mathbf{y}^{\text{hard}} \leftarrow \mathbf{0}^N$ ▷ All anchors are real

**return** $\{(\tilde{\mathbf{x}}_k, \tilde{y}_k, 0)\}_{k=1}^{N}$

**Key difference from Alg. 4:** No RR/FF/FR pairs — every output is an RF-mixed image. Batch size halves from $2N$ (loaded) to $N$ (mixed).

**Algorithm 6** Loss Computation

**Require:** Logits $\mathbf{P} \in \mathbb{R}^{B\times 2}$; Soft labels $\tilde{\mathbf{y}} \in [0,1]^B$ (if mixup active) or hard labels $\mathbf{y} \in \{0,1\}^B$
**Ensure:** Loss dict $\{\text{overall}, \text{real\_loss}, \text{fake\_loss}\}$

**// Primary loss: soft-label or hard-label cross-entropy**
**if** $\tilde{\mathbf{y}}$ is provided (mixup active) **then**
  $\mathbf{L}_{\text{ce}} \leftarrow -\big[\tilde{\mathbf{y}} \cdot \log \text{softmax}(\mathbf{P})_1 + (1-\tilde{\mathbf{y}}) \cdot \log \text{softmax}(\mathbf{P})_0\big]$
  $\mathcal{L}_{\text{overall}} \leftarrow \frac{1}{B}\sum_{i=1}^{B} L_{\text{ce},i}$
  **// Per-class decomposition via soft-label weighting**
  $w_{\text{real}} \leftarrow \sum_i (1-\tilde{y}_i); \quad w_{\text{fake}} \leftarrow \sum_i \tilde{y}_i$
  $\mathcal{L}_{\text{real}} \leftarrow \frac{1}{\max(w_{\text{real}},1)} \sum_i (1-\tilde{y}_i) \cdot L_{\text{ce},i}$
  $\mathcal{L}_{\text{fake}} \leftarrow \frac{1}{\max(w_{\text{fake}},1)} \sum_i \tilde{y}_i \cdot L_{\text{ce},i}$
**else**
  $\mathbf{L}_{\text{ce}} \leftarrow \text{CrossEntropy}(\mathbf{P}, \mathbf{y})$
  $\mathcal{L}_{\text{overall}} \leftarrow \frac{1}{B}\sum_{i=1}^{B} L_{\text{ce},i}$
  **// Per-class decomposition via hard labels**
  $\mathcal{L}_{\text{real}} \leftarrow \frac{1}{|\{i:y_i=0\}|} \sum_{i:y_i=0} \text{CE}(\mathbf{P}_i, y_i)$
  $\mathcal{L}_{\text{fake}} \leftarrow \frac{1}{|\{i:y_i=1\}|} \sum_{i:y_i=1} \text{CE}(\mathbf{P}_i, y_i)$
**end if**
**return** $\{\text{overall} : \mathcal{L}_{\text{overall}},\ \text{real\_loss} : \mathcal{L}_{\text{real}},\ \text{fake\_loss} : \mathcal{L}_{\text{fake}}\}$

**Algorithm 7** Training Loop (per epoch)

**Require:** Training set $\mathcal{D}_{\text{train}}$; validation sets $\{\mathcal{D}_{\text{val}}^k\}$; model $F = g_\phi \circ f_\theta$; optimizer $\mathcal{O}$; mixup config $\mathcal{M}$; sampler config $\mathcal{S}$; $E$ epochs; eval interval $T$ (steps)
**Ensure:** Best model checkpoint by primary metric (AUC)

**for** epoch $= 1$ **to** $E$ **do**
  **for** each batch $\mathcal{B} = \{(\mathbf{x}_i, y_i)\} \subset \mathcal{D}_{\text{train}}$ **do**
    **// Step 1: On-the-fly mixup augmentation (training only)**
    **if** $\mathcal{M}$.use_mixup **then**
      **if** $\mathcal{S}$.v2 (BalancePairSampler) **then**
        $\{(\tilde{\mathbf{x}}_k, \tilde{y}_k, 0)\} \leftarrow$ RFPairLapPyramidMixup($\mathcal{B}$) ▷ Alg. 5
      **else**
        $\{(\tilde{\mathbf{x}}_i, \tilde{y}_i, y_i^{\text{hard}})\} \leftarrow$ LapPyramidMixup($\mathcal{B}$) ▷ Alg. 4
      **end if**
    **end if**
    **// Step 2: Forward pass**
    $\mathbf{Z} \leftarrow f_\theta(\tilde{\mathbf{X}})$;  $\mathbf{P} \leftarrow g_\phi(\mathbf{Z})$
    **// Step 3: Loss computation (Alg. 6)**
    $\mathbf{L} \leftarrow$ LossComputation($\mathbf{P}, \tilde{\mathbf{y}}$)
    **// Step 4: Standard backward pass**
    $\mathcal{O}$.zero_grad()
    $\mathbf{L}$.overall.backward()
    $\mathcal{O}$.step()
    **// Step 5: Periodic evaluation**
    **if** global_step mod $T = 0$ **then**
      **for** each $\mathcal{D}_{\text{val}}^k$ **do**
        metrics$_k \leftarrow$ Evaluate($F, \mathcal{D}_{\text{val}}^k$) ▷ No mixup; $\hat{y}_i = \text{softmax}(F(\mathbf{x}_i))_1$ vs. $y_i$
        Update best ckpt if AUC improves
      **end for**
    **end if**
    global_step $\leftarrow$ global_step $+ 1$
  **end for**
**end for**
**return** best checkpoint